

1 - Estructuras Series y DataFrame: conceptos y transformaciones

Introducción a las estructuras fundamentales de Pandas: Series y DataFrame. Comprender sus características, diferencias y cómo realizar operaciones básicas y transformaciones vectorizadas para manipular datos tabulares de manera eficiente en Python.

Introducción

Imagina que tienes un conjunto de datos con información sobre ventas, clientes o sensores, y necesitas analizarlo rápidamente para tomar decisiones informadas. ¿Cómo organizarías esos datos para facilitar su manipulación y análisis? Aquí es donde entran en juego las estructuras de datos de Pandas: Series y DataFrame.



En esta unidad, exploraremos estas dos estructuras fundamentales que permiten manejar datos tabulares y unidimensionales con etiquetas, facilitando operaciones complejas de forma sencilla y eficiente. Aprenderás a crear, seleccionar y transformar datos usando estas estructuras, habilidades esenciales para cualquier científico de datos.

Al finalizar, podrás distinguir claramente entre Series y DataFrame, y aplicar operaciones básicas y transformaciones vectorizadas para preparar tus datos para análisis más avanzados. Esta base te preparará para trabajar con múltiples fuentes de datos y combinarlas en las siguientes unidades.

Teoría

¿Qué es Pandas?

Pandas es una biblioteca de Python diseñada para la manipulación y análisis de datos, especialmente datos tabulares. Proporciona estructuras de datos flexibles y potentes que facilitan el trabajo con datos heterogéneos y etiquetados.

Series

Una Series es una estructura unidimensional similar a un arreglo o lista, pero con etiquetas (índices) para cada elemento. Esto permite acceder a los datos no solo por posición, sino también por etiqueta.

Características clave:

- Unidimensional
- Índices etiquetados
- Puede contener datos de cualquier tipo (enteros, cadenas, flotantes, etc.)

```
import pandas as pd

s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])

print(s['b'])

# Salida: 20
```

DataFrame

Un DataFrame es una estructura bidimensional, similar a una tabla o una hoja de cálculo, compuesta por filas y columnas, donde cada columna es una Series.

Características clave:

- Bidimensional

- Índices para filas y columnas
- Cada columna puede tener un tipo de dato diferente

```
df = pd.DataFrame({
    'Edad': [25, 30, 22],
    'Nombre': ['Ana', 'Luis', 'Carlos']})
print(df['Edad'])
# Devuelve una Series con las edades
```

Diferencias entre Series y DataFrame

Característica	Series	DataFrame
Dimensionalidad	1D	2D
Estructura	Vector con índice	Tabla con filas y columnas
Uso típico	Columna única o vector de datos	Conjunto de datos tabular

Operaciones básicas y transformaciones

Pandas permite realizar operaciones vectorizadas, es decir, aplicar funciones a toda una columna o fila sin necesidad de bucles explícitos, lo que mejora la eficiencia.

Ejemplos:

- Selección de columnas:

```
python
columna = df['Edad']
```

- Filtrado por condición:

```
python
mayores_25 = df[df['Edad'] > 25]
```

- Transformación vectorizada:
-

```
python  
df['Edad_doble'] = df['Edad'] * 2
```

Estas operaciones son fundamentales para preparar y limpiar datos antes de análisis más complejos.

Contexto y Aplicaciones Prácticas

En la industria de la ciencia de datos, manejar datos tabulares es una tarea diaria. Por ejemplo, en análisis de ventas, cada fila puede representar una transacción y cada columna un atributo como fecha, producto o monto. Pandas facilita:

- **Organizar datos heterogéneos:** Combinar datos numéricos, categóricos y fechas en un solo DataFrame.
- **Acceso eficiente:** Seleccionar columnas específicas para análisis o visualización.
- **Transformaciones rápidas:** Crear nuevas columnas derivadas, filtrar datos relevantes y preparar conjuntos para modelos predictivos.

Este conocimiento es la base para leer datos desde archivos CSV, TXT o Excel, y combinarlos, que veremos en unidades posteriores. Además, permite construir flujos reproducibles de análisis exploratorio, clave para proyectos profesionales en ciencia de datos.

2 - Diagnóstico de calidad de datos y detección de valores faltantes

En esta unidad, exploraremos cómo identificar y diagnosticar problemas comunes en datasets reales, con especial énfasis en la detección y tratamiento de valores ausentes. Aprenderás metodologías prácticas para evaluar la calidad de los datos y preparar un flujo reproducible de análisis exploratorio (EDA) que facilite la limpieza y generación de insights confiables.

Diagnóstico de calidad de datos: ¿Por qué es crucial en Ciencia de Datos?

Imagina que recibes un conjunto de datos para un proyecto de análisis, pero al comenzar a explorarlo, notas que hay valores faltantes, inconsistencias y posibles errores. ¿Cómo decides qué

problemas abordar primero? ¿Cómo puedes garantizar que el análisis posterior sea confiable y reproducible?

En esta unidad, te sumergirás en las metodologías para diagnosticar la calidad de datos, enfocándote en la detección de valores ausentes y otros problemas comunes en datasets tabulares. Además, aprenderás a construir un flujo reproducible de análisis exploratorio (EDA) que te permitirá documentar y automatizar la limpieza y exploración de datos, una habilidad esencial para cualquier científico de datos profesional.

Este conocimiento te permitirá identificar y priorizar acciones de limpieza, asegurando que tus análisis sean sólidos y tus resultados, confiables. Además, conectarás estos conceptos con lo aprendido previamente, preparando el terreno para las siguientes unidades donde profundizaremos en técnicas avanzadas de tratamiento y visualización.



Diagnóstico de calidad de datos: conceptos clave

Datos ausentes y técnicas de tratamiento (Introducción)

Los *datos ausentes* o *valores faltantes* son entradas que no contienen información en un dataset. Pueden aparecer por diversas razones: errores en la recolección, problemas de transmisión, o simplemente porque la información no aplica.

¿Por qué es importante detectarlos? Porque pueden sesgar análisis y modelos si no se manejan adecuadamente.

Técnicas básicas para tratar datos ausentes:

- **Identificación:** Uso de funciones como `isnull()` o `isna()` en Pandas para detectar valores faltantes.
- **Eliminación:** Remover filas o columnas con valores ausentes, útil cuando la cantidad es pequeña y no afecta la representatividad.
- **Imputación:** Sustituir valores faltantes por estimaciones, como la media, mediana, moda o valores calculados según contexto.
- **Sustitución basada en contexto:** Por ejemplo, rellenar con un valor específico que tenga sentido para la variable (como 0 para conteos).

***Nota:** La elección de la técnica depende del contexto y del impacto que los datos ausentes puedan tener en el análisis.*

Metodologías para diagnóstico: conteo de nulos y detección de outliers

- **Conteo de nulos:** Es fundamental cuantificar cuántos valores faltan por columna o fila para evaluar la magnitud del problema.

Python

```
import pandas as pd
df = pd.read_csv('dataset.csv')print(df.isnull().sum())
```

- **Detección de outliers:** Valores atípicos que pueden indicar errores o casos especiales. Métodos comunes incluyen:
 - Estadísticas descriptivas (media, desviación estándar)
 - Boxplots
 - Z-score

Análisis exploratorio reproducible (EDA) (Introducción)

Un flujo reproducible de EDA implica documentar y automatizar el proceso de limpieza, visualización y generación de insights usando notebooks (Jupyter, Google Colab). Esto garantiza que cualquier persona pueda seguir y replicar el análisis.

Elementos clave:

- Código limpio y comentado
- Uso de funciones para tareas repetitivas
- Documentación clara de decisiones de limpieza
- Visualizaciones que apoyen la comprensión

Ejemplo: Un notebook que carga datos, detecta valores ausentes, los trata según estrategia definida, y genera gráficos para entender la distribución.

Conocimiento previo: Recuerda que ya has trabajado con estructuras de datos en Pandas y técnicas básicas de manipulación (módulo 3). Ahora profundizaremos en cómo diagnosticar problemas para preparar un análisis confiable.

Aplicación práctica en proyectos de Ciencia de Datos

En la industria, los datasets raramente están limpios. Por ejemplo, en análisis de calidad de vinos, es común encontrar registros con valores faltantes o erróneos que pueden afectar la evaluación de la calidad.

Un diagnóstico efectivo permite:

- Priorizar qué columnas o filas requieren limpieza urgente.
- Decidir si es mejor eliminar o imputar datos ausentes según el impacto en el análisis.
- Documentar el proceso para que otros científicos de datos o stakeholders puedan entender y reproducir el análisis.

Además, construir un flujo reproducible de EDA facilita la colaboración y la actualización del análisis cuando se reciben nuevos datos o se ajustan hipótesis.

Este enfoque profesional es estándar en equipos de ciencia de datos y es fundamental para garantizar la calidad y confiabilidad de los resultados.

3 - Qué es ndarray y por qué usar NumPy

Introducción a la estructura ndarray de NumPy, sus ventajas en cómputo numérico, tipos de datos (dtypes) y cómo estos afectan la memoria y el rendimiento. Se comparan arrays de NumPy con listas de Python para destacar beneficios prácticos en ciencia de datos.



¿Por qué NumPy y su estructura ndarray son esenciales en Ciencia de Datos?

Imagina que tienes que procesar millones de datos numéricos para un análisis estadístico o un modelo predictivo. ¿Usarías listas de Python tradicionales para esta tarea? Probablemente no, porque aunque las listas son flexibles, no están optimizadas para cálculos numéricos intensivos. Aquí es donde entra en juego ndarray, la estructura fundamental de NumPy, diseñada para manejar datos numéricos de forma eficiente y rápida.

En esta unidad, descubrirás qué es un ndarray, cómo almacena los datos en memoria y por qué esto es crucial para el rendimiento. También exploraremos los tipos de datos (dtypes) que puedes usar y cómo afectan la precisión y el uso de memoria. Al finalizar, podrás explicar las ventajas de ndarray y reconocer los tipos de datos más comunes en NumPy, sentando las bases para manipular datos numéricos con eficacia en Python.

¿Qué es un ndarray?

Un ndarray (N-dimensional array) es la estructura de datos principal de NumPy para almacenar arreglos de datos numéricos en una o más dimensiones. A diferencia de las listas de Python, que pueden contener elementos heterogéneos y están almacenadas como objetos separados en memoria, un ndarray almacena datos homogéneos (todos del mismo tipo) en un bloque contiguo de memoria.

Características clave de ndarray:

- **Homogeneidad:** Todos los elementos tienen el mismo tipo de dato (dtype).
- **Memoria contigua:** Los datos están almacenados en bloques continuos, lo que mejora la velocidad de acceso y operaciones.
- **Shape:** Define la forma del array, es decir, cuántas dimensiones tiene y el tamaño en cada una.
- **Operaciones vectorizadas:** Permite realizar operaciones matemáticas sobre arrays completos sin necesidad de bucles explícitos.

***Ejemplo:** Un array 2D con shape (3, 4) tiene 3 filas y 4 columnas.*

Tipos de datos (dtypes) y su impacto

El dtype define el tipo de cada elemento en el array, por ejemplo:

Tipo de dato	Descripción	Tamaño en memoria
int32	Entero de 32 bits	4 bytes
float64	Número de punto flotante 64 bits	8 bytes
bool	Booleano	1 byte

El tipo de dato afecta:

- **Precisión:** Por ejemplo, float64 es más preciso que float32.
- **Uso de memoria:** Tipos más grandes consumen más memoria.
- **Performance:** Operaciones con tipos más pequeños suelen ser más rápidas.

Strides

Los *strides* indican cuántos bytes hay que saltar en memoria para pasar de un elemento al siguiente en cada dimensión. Esto es importante para entender cómo NumPy accede a los datos internamente y optimiza las operaciones.

Nota: Aunque no profundizaremos en strides ahora, es un concepto clave para entender la eficiencia de NumPy.

Comparación con listas de Python

Característica	Listas de Python	ndarray de NumPy
Tipo de datos	Heterogéneo	Homogéneo
Almacenamiento	Objetos dispersos en memoria	Bloque contiguo de memoria
Operaciones vectorizadas	No soportadas nativamente	Sí, muy eficientes
Uso de memoria	Mayor debido a objetos	Menor y más eficiente

En la práctica profesional de ciencia de datos, trabajar con grandes volúmenes de datos numéricos es común. Usar estructuras de datos ineficientes puede ralentizar el análisis y consumir recursos innecesarios.

Por ejemplo, al analizar datos de sensores, imágenes o series temporales, la rapidez en el acceso y procesamiento de datos es crucial. NumPy, con su ndarray, permite realizar cálculos vectorizados que aprovechan la memoria contigua y tipos de datos optimizados, acelerando procesos que con listas tradicionales serían prohibitivos.

Además, entender los dtypes es fundamental para balancear precisión y uso de memoria, especialmente cuando se trabaja con datasets grandes o en entornos con recursos limitados. Esta unidad te prepara para crear y manipular arrays de NumPy, base para operaciones más avanzadas que veremos en unidades posteriores, como álgebra lineal optimizada y manipulación avanzada de datos.

4 - Álgebra lineal con NumPy

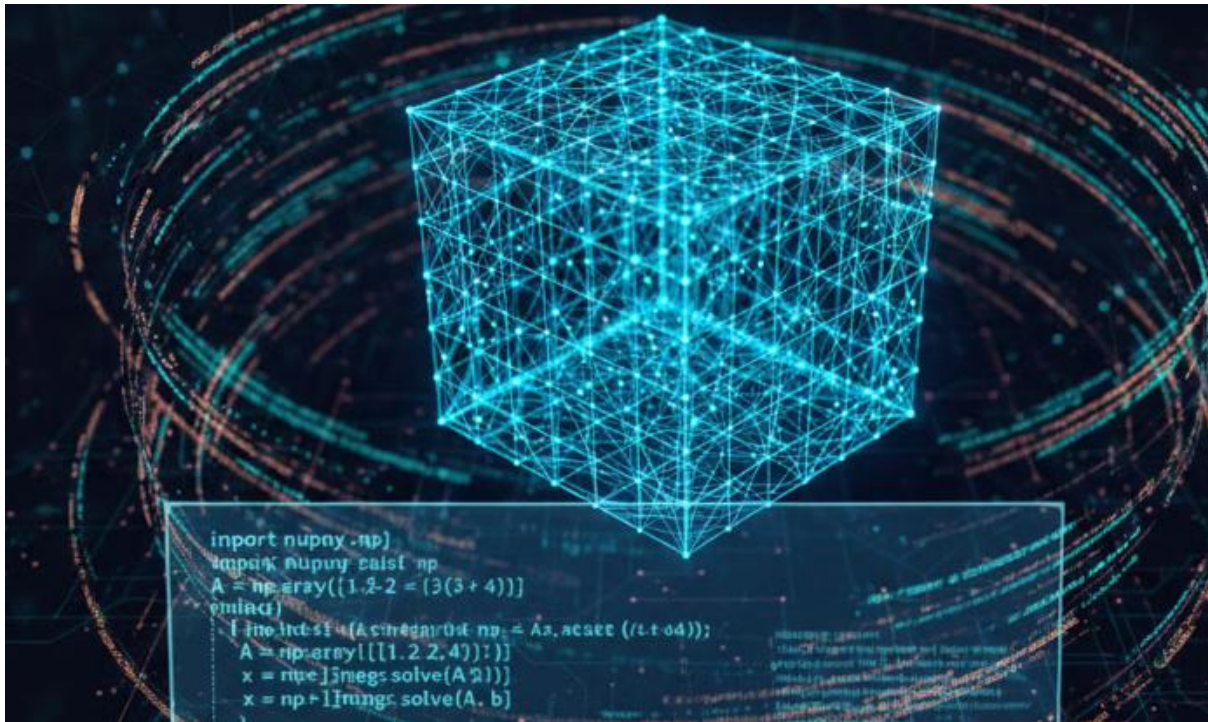
Introducción práctica a operaciones de álgebra lineal con NumPy, enfocada en multiplicaciones matriciales, producto punto, transpuestas y solución de sistemas lineales usando `np.linalg.solve`, con ejemplos aplicados y casos de uso en ciencia de datos.

Álgebra lineal con NumPy: Potenciando el análisis numérico

Imagina que tienes un conjunto de datos con múltiples variables y necesitas realizar transformaciones complejas o resolver sistemas de ecuaciones para modelar relaciones entre ellas. ¿Cómo hacerlo de forma eficiente y reproducible? Aquí es donde el álgebra lineal y NumPy se convierten en tus mejores aliados.

En esta unidad, exploraremos cómo NumPy facilita operaciones matriciales optimizadas, desde multiplicaciones hasta la solución de sistemas lineales, herramientas esenciales para cualquier científico de datos que busque manejar grandes volúmenes de información con rapidez y precisión.

Al finalizar, serás capaz de identificar cuándo aplicar estas operaciones y utilizar funciones clave de `np.linalg` para resolver problemas reales, sentando las bases para análisis más avanzados y optimizaciones en tus proyectos.



Fundamentos de Álgebra Lineal en NumPy

1. Operaciones Matriciales y Vectorizadas

En NumPy, las matrices y vectores se representan mediante arrays multidimensionales (ndarray). Las operaciones matriciales se realizan de forma vectorizada, lo que significa que se ejecutan a nivel de bloques de datos, optimizando el rendimiento.

- **Multiplicación matricial:** Se realiza con la función `np.dot()` o el operador `@`.

```
python
import numpy as np
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
C = np.dot(A, B)
# o C = A @ B
```

- **Producto punto:** Es un caso particular de multiplicación para vectores, que devuelve un escalar.

```
python
v1 = np.array([1, 2, 3])
v2 = np.array([4, 5, 6])
dot_product = np.dot(v1, v2) # Resultado: 32
```

2. Transpuesta de Matrices

La transpuesta de una matriz intercambia sus filas por columnas, útil para operaciones como el cálculo de productos internos o la preparación de datos.

```
python
A_T = A.T
```

3. Solución de Sistemas Lineales con `np.linalg.solve`

Resolver sistemas de ecuaciones lineales es común en modelado y análisis. NumPy

ofrece `np.linalg.solve(A, b)` para encontrar el vector x que satisface $Ax = b$.

```
python
A = np.array([[3, 1], [1, 2]])
b = np.array([9, 8])
x = np.linalg.solve(A, b) # Solución del sistema
```

Esta función es más eficiente y numéricamente estable que calcular la inversa de A y multiplicar.

4. Funciones de Álgebra Lineal en NumPy

El módulo `np.linalg` incluye funciones para:

- Determinantes (`np.linalg.det`)
- Valores y vectores propios (`np.linalg.eig`)
- Normas y distancias (`np.linalg.norm`)

Estas herramientas son fundamentales para análisis estadísticos y optimización.

***Nota:** Siempre verifica que las matrices sean cuadradas y no singulares antes de aplicar estas funciones para evitar errores.*

Aplicaciones Prácticas en Ciencia de Datos

Las operaciones de álgebra lineal con NumPy son la base para:

- **Transformaciones de datos:** Rotaciones, escalados y proyecciones en análisis exploratorio.
- **Modelado estadístico:** Ajuste de modelos lineales y regresiones.
- **Optimización:** Cálculo eficiente de gradientes y soluciones en problemas de optimización.

Por ejemplo, al trabajar con grandes conjuntos de datos, resolver sistemas lineales permite ajustar modelos predictivos sin recurrir a métodos iterativos lentos.

Además, la capacidad de realizar multiplicaciones matriciales optimizadas acelera cálculos en pipelines de procesamiento y análisis, especialmente cuando se integran con Pandas para manipulación tabular posterior.

Esta unidad prepara el terreno para aplicar estas técnicas en flujos de trabajo reproducibles y escalables, fundamentales en proyectos reales de ciencia de datos.

5 - Operaciones básicas y broadcasting con arrays

Ejercicio para aplicar operaciones aritméticas vectorizadas y broadcasting en arrays de diferentes dimensiones sin usar bucles explícitos.

¿Qué es broadcasting?

Broadcasting es la regla que permite operar arrays de distintas formas sin escribir bucles: NumPy “ajusta” dimensiones menores para que coincidan con dimensiones mayores, siguiendo estas reglas básicas:

1. Si los arrays tienen distinta cantidad de dimensiones, se **pre-pend** 1s a la forma del array de menor dimensión.
2. Luego, para cada dimensión (de derecha a izquierda), las dimensiones son compatibles si son iguales o una de ellas es 1.
3. Si una dimensión es 1 en un array, ese array se considera “repetido” a lo largo de esa dimensión para que la operación pueda realizarse (sin realmente replicar en memoria).

Casos típicos:

- Escalar + array (el escalar se “extiende” a todo el array).
- Array 1D con longitud m + array 2D (n, m) (la 1D actúa sobre filas).
- Vector columna $(n,1)$ + matriz (n,m) (el vector se “empaqueta” por columnas).

Ventajas:

- Código claro y compacto.
- Mucho más rápido que bucles en Python (operaciones vectorizadas en C).

Buenas prácticas:

- Asegurarse que los shapes sean compatibles; en dudas, inspeccionar `arr.shape`.
- Evitar operaciones que desembocan en copias innecesarias si el dataset es muy grande.
- Para operaciones in-place, usar máscaras booleanas para no crear arrays intermedios grandes innecesarios.

6 - Ejercicio práctico: Lectura de csv en Pandas

Ejercicio para practicar la lectura y escritura de datos usando pandas con archivos CSV, TXT y Excel. Se evaluará la capacidad para importar datos con parsing correcto, manejo de tipos y escritura de resultados.

En este ejercicio, pondremos en práctica la lectura y escritura de datos usando pandas, enfocándonos en archivos CSV, TXT y Excel. Aprenderás a manejar correctamente el parsing, la conversión de tipos y la escritura de resultados en archivos.

Descripción del problema

Se te proporciona un conjunto de datos en diferentes formatos (CSV, TXT, Excel). Tu tarea es leer estos datos, realizar un procesamiento simple y escribir el resultado en un archivo CSV.

Enlace para descargar los archivos:

<https://www.kaggle.com/datasets/hanicier/master-pokemon-dataset-with-corpus-version>

<https://www.kaggle.com/datasets/uciml/red-wine-quality-cortez-et-al-2009>

<https://www.kaggle.com/datasets/gpreda/life-expectancy-in-europe-by-age-and-sex>

Detalles:

- El primer archivo es un CSV con datos numéricos y categóricos.
- El segundo archivo es un TXT con datos tabulados.
- El tercer archivo es un Excel con varias hojas.

Debes:

1. Leer cada archivo correctamente, especificando el encoding y delimitadores cuando sea necesario.
2. Convertir las columnas a los tipos adecuados (por ejemplo, numéricos, categóricos).
3. Combinar los datos en un solo DataFrame según se indique.

4. Realizar un cálculo simple: por ejemplo, calcular la media de una columna numérica agrupada por una categoría.
5. Escribir el resultado en un archivo CSV.

Nota: En este ejercicio no se requiere que el programa cree o modifique archivos en disco, solo que procese la entrada y produzca la salida solicitada en stdout en formato CSV.

7 - Tratamiento de datos ausentes: eliminación e imputación

Ejercicio para aplicar técnicas de tratamiento de datos ausentes en un conjunto de datos simulado.

El objetivo es decidir cuándo eliminar filas con datos faltantes y cuándo imputar valores faltantes usando estrategias simples, y comparar resultados.

En este ejercicio, recibirás un conjunto de datos tabular con valores numéricos y algunos valores ausentes indicados con la palabra NA.

Tu tarea es implementar un programa que:

- Detecte las filas que contienen valores ausentes.
- Decida si eliminar la fila o imputar los valores ausentes según una regla simple:
- Si la fila tiene más de un valor ausente, eliminarla.
- Si la fila tiene exactamente un valor ausente, imputar ese valor con la media de esa columna (calculada solo con los valores presentes).

Finalmente, el programa debe imprimir el conjunto de datos resultante tras aplicar estas transformaciones.

1. Contexto del problema

En problemas reales de Data Science es muy frecuente encontrar datos incompletos: valores faltantes por errores de carga, fallas en sensores, problemas de integración entre sistemas o simplemente información no disponible al momento de la medición.

El tratamiento de estos valores ausentes es una etapa crítica del preprocesamiento, ya que impacta directamente en:

- La calidad de los análisis exploratorios
- El entrenamiento de modelos de Machine Learning

- La reproducibilidad y confiabilidad de los resultados

En esta práctica trabajaremos con un dataset comercial simulado, que representa métricas de negocio (por ejemplo ventas, unidades, ingresos) asociadas a distintos tipos de clientes. Algunas de estas métricas contienen valores ausentes (NA), que deben ser tratados siguiendo reglas claras y reproducibles.

2. Objetivo

El objetivo principal es diseñar y aplicar una estrategia simple y justificada para el tratamiento de datos ausentes, combinando:

- **Eliminación de filas** cuando la información perdida es excesiva
- **Imputación por la media** cuando la pérdida de información es mínima

Al finalizar la práctica, deberás obtener un conjunto de datos limpio, coherente y listo para ser utilizado en análisis posteriores.

3. Identificación de valores ausentes

El primer paso consiste en detectar la presencia de datos ausentes.

En esta práctica, los valores ausentes están representados internamente como NaN (Not a Number), que es el estándar utilizado por pandas y numpy.

Se debe analizar:

- **Por fila:** cuántos valores ausentes tiene cada registro
- **Por columna:** cuántos valores ausentes tiene cada variable numérica

Este análisis permite tomar decisiones informadas sobre cómo tratar cada fila del dataset.

4. Regla de decisión: eliminar o imputar

No todos los valores ausentes deben tratarse de la misma manera. En este ejercicio se aplica la siguiente regla simple:

● Eliminación de filas

- Si una fila tiene **más de un valor ausente**, se considera que perdió demasiada información.
- En ese caso, la fila se **elimina completamente** del dataset.

Justificación:

Con múltiples valores faltantes, la imputación podría introducir demasiado ruido o sesgo, afectando la calidad de los datos.

□ Imputación de valores

- Si una fila tiene **exactamente un valor ausente**, ese valor se **imputa**.
- El valor imputado corresponde a la **media de la columna** a la que pertenece el dato faltante.

Justificación:

Cuando la pérdida de información es mínima (un solo valor), imputar permite conservar el registro sin distorsionar significativamente la distribución de la variable.

5. Cálculo correcto de las medias

Un aspecto clave del ejercicio es cuándo calcular las medias:

- Las medias de cada columna deben calcularse **antes de eliminar filas**
- Solo deben considerarse los **valores presentes** (ignorando los NA)

Esto garantiza que:

- Las estadísticas usadas para imputar no estén sesgadas por decisiones posteriores
- El proceso sea **reproducible y auditable**

6. Aplicación de las transformaciones

Una vez definidas las reglas y calculadas las estadísticas necesarias, el pipeline lógico es:

1. Identificar filas con más de un valor ausente y eliminarlas
2. Identificar filas con exactamente un valor ausente
3. Reemplazar ese valor faltante por la media correspondiente
4. Verificar que el dataset resultante no contenga valores ausentes en las columnas numéricas

Este enfoque separa claramente la lógica de decisión de la aplicación de los cambios, una buena práctica en pipelines de datos.

7. Formato de salida y reproducibilidad

El resultado final debe imprimirse siguiendo un formato estricto:

- Solo columnas numéricas
- Valores separados por espacios

- Cada valor con **dos decimales**
- Sin encabezados

Además, se solicita registrar información clave del proceso (cantidad de filas eliminadas, imputadas, medias utilizadas), lo que permite:

- Auditar el tratamiento realizado
- Repetir el experimento con los mismos resultados
- Facilitar la evaluación automática

8 - Combinar y agregar datos: merge, join y groupby

Ejercicio para practicar la combinación y agregación de datos usando técnicas similares a merge, join y groupby. Se reciben dos conjuntos de datos con claves y valores, y se debe unir y agregar la información según instrucciones específicas.

EDA avanzado, limpieza, imputación y normalización con Python

En la práctica real de Data Science rara vez trabajamos con un único fichero “limpio”. Normalmente combinamos tablas (clientes, productos, transacciones), resumimos y agregamos para responder preguntas de negocio. Saber unir correctamente y agregar con sentido es clave para evitar errores (duplicados, sesgos, dobles conteos) y para construir pipelines reproducibles.

1) Combinar datos — conceptos esenciales

Clave (key)

Las uniones se basan en una(s) columna(s) compartida(s) que identifican correspondencias (p. ej. customer_id, order_id). Asegúrate que las claves sean del mismo tipo (strings / ints) y que los valores no estén en formatos incompatibles.

Tipos de join (resumen conceptual)

- **Inner join:** conserva solo las filas con key presente en ambas tablas. Útil cuando necesitas la intersección.
- **Left join:** conserva todas las filas de la izquierda; agrega información de la derecha cuando exista. Muy usado para enriquecer una tabla principal.
- **Right join:** simétrico del left (menos usado).

- **Outer (full) join:** conserva todas las filas de ambas tablas, con NaN donde no hay match. Sirve para auditoría de faltantes.

Casos a considerar

- **Keys duplicadas:** si la clave no es única en una tabla (por ejemplo, varios pedidos por cliente), la unión puede multiplicar filas (producto cartesiano entre coincidencias). Esto puede ser deseado (detalle-por-detalle) o un error.
- **NaN en keys:** pandas normalmente no empareja NaN; decide si imputarlos o filtrarlos antes del merge.
- **Tipos diferentes:** 1 (int) ≠ '1' (str) — conviene homogeneizar.

Buenas prácticas al hacer merge en pandas

- Usar `pd.merge(left, right, on='key', how='left', validate='one_to_many')` para hacer explícito el contrato de unicidad; `validate` lanza error si el contrato no se cumple.
- Cuando hay columnas con mismo nombre, usar `suffixes=('_left','_right')`.
- Para joins por índice: `left.join(right, how='left')` o `right.join(...)` después de `.set_index(...)`.
- Evitar merges por columnas con tipo float (riesgo de imprecisión).

Ejemplo mínimo (ilustrativo):

```
# enriquecer orders con nombre de cliente
orders = pd.merge(orders,
customers[['customer_id', 'name']], on='customer_id', how='left',
validate='many_to_one')
```

2) Agregar datos — groupby (concepto y patrones)

Idea central

`groupby` = *split-apply-combine*:

1. **Split:** dividir filas por valores únicos de una o más columnas (p. ej. `category`).
2. **Apply:** calcular una función agregada por grupo (media, suma, conteo).
3. **Combine:** recomponer el resultado en un DataFrame compacto.

Operaciones comunes

- `df.groupby('category')['sales'].sum()` — total por categoría.
- `df.groupby(['region','category']).agg({'sales':'sum','units':'mean'})` — agregación múltiple por grupo.
- `as_index=False` evita que `groupby` devuelva índices jerárquicos por defecto: `df.groupby('category', as_index=False)['sales'].sum()`.

Transform vs Aggregate

- `agg` / `aggregate`: produce una salida reducida (una fila por grupo).

- `transform`: devuelve una serie del mismo tamaño que el original, útil para crear columnas normalizadas por grupo (p. ej. `df['sales_z'] = df.groupby('category')['sales'].transform(lambda x: (x-x.mean())/x.std())`).

Filtrar grupos

- `df.groupby('category').filter(lambda g: g['sales'].sum() > 1000)` — conservar solo grupos que cumplan condición.

Pivot / `pivot_table`

- Para transformar agregados a un formato ancho (tabla cruzada):

```
df.pivot_table(index='region', columns='category', values='sales', aggfunc='sum', fill_value=0).
```

Ejemplo práctico resumido:

```
agg = df.groupby(['month', 'category']).agg(    total_sales = ('sales', 'sum'),
avg_price   = ('price', 'mean'),    n_orders   =
('order_id', 'nunique')).reset_index()
```

3) Errores y trampas comunes (y cómo detectarlos)

- **Duplicación inesperada de filas** tras un merge → revisar si las claves en alguna tabla no son únicas.
Solución: `right[~right['key'].duplicated(keep=False)]` o usar `validate` para detectar.
- **Dobles conteos al agregar** si antes no se deduplicó a la entidad correcta (por ejemplo, sumar `order_line.total` y luego sumar por `order_id` sin agrupar puede contar dos veces).
- **Nulos ocultos**: valores codificados como `-1`, `"` o `'NA'` no se reconocen como `NaN` hasta que los conviertas.
- **Merge silencioso que “pierde” datos**: usar `indicator=True` en `pd.merge(..., indicator=True)` para entender cuántas filas vienen de cada lado (`left_only`, `right_only`, `both`).

4) Rendimiento y escala (tips prácticos)

- Para tablas grandes:
 - Indexar las columnas de join (`.set_index('key')`) y usar `join`, o asegurarte que las columnas son `Categorical` si tienen muchas repeticiones.
 - Evitar `apply` row-wise; preferir operaciones vectorizadas y agregaciones nativas.
 - Usar `pd.concat` con listas de DataFrames cuando se ensamblan muchas partes.
 - Considerar herramientas como Dask o databases si el dataset ya excede memoria.
-

5) Idiomas y estilo Pythonic (pequeñas recomendaciones)

- Prefiere funciones pequeñas y con **contrato claro** (inputs, outputs, no mutar argumentos a menos que se diga).
- Usa `validate` en merges y `assert` para checks rápidos.
- Prefiere `agg` con diccionarios legibles en vez de cadenas mágicas.
- Evita `inplace=True` en pandas por claridad y por facilidades de debugging.

1. Contexto y objetivo general

En proyectos reales de Data Science, la mayor parte del tiempo no se dedica al modelado, sino a comprender, limpiar y preparar los datos.

El objetivo de esta práctica es simular un flujo de trabajo profesional de Exploratory Data Analysis (EDA) sobre un dataset de negocio realista, incorporando:

- Análisis exploratorio estructurado
- Visualizaciones interactivas
- Identificación de problemas de calidad de datos
- Limpieza, imputación y tratamiento de outliers
- Normalización de variables numéricas
- Generación de salidas reproducibles

Todo el proceso se diseña para ser determinístico, reproducible y ejecutable en una única celda, tal como se espera en entornos productivos o pipelines automatizados.

2. Dataset simulado: realismo y control experimental

El dataset representa **ventas de una empresa multirubro**, con variables típicas de un sistema comercial:

- Variables **categorías reales**:
- `categoria` (Alimentos, Bebidas, Electrónica, etc.)
- `region` (Norte, Sur, Este, Oeste, Centro)
- Variables **numéricas de negocio**:
- `ventas`, `unidades`, `margen_pct`, `dias_stock`, `satisfaction`, `ingresos_estim`
- Variables **temporales**:
- `fecha`

Además, el dataset incorpora **problemas comunes del mundo real** de forma controlada:

- Valores faltantes codificados como -1
- Duplicados
- Outliers extremos
- Distribuciones no normales (ventas log-normales)

Esto permite practicar técnicas de EDA **sin depender de fuentes externas** y sin introducir incertidumbre sobre la calidad del dato original.

3. EDA básico: entender antes de transformar

El primer bloque del EDA responde a una pregunta clave:

¿Qué datos tengo y en qué estado están?

Por eso se analizan:

- **Estructura del dataset:** columnas y tipos de datos
- **Estadísticos descriptivos:** media, desviación, percentiles
- **Valores inválidos:** conteo de -1 por columna
- **Duplicados:** impacto en el tamaño efectivo del dataset
- **Percentiles extremos:** para anticipar la presencia de outliers

Este paso es crítico porque **ninguna transformación posterior es válida si no se comprende el estado inicial de los datos**.

4. Visualizaciones interactivas: detectar patrones y anomalías

Las visualizaciones permiten **detectar problemas que no se ven en tablas**. En esta práctica se utilizan gráficos coherentes con análisis profesional:

Histograma de ventas

Permite observar:

- Asimetría
- Cola larga
- Impacto de outliers

- Distribución real de ingresos

Boxplots de variables numéricas

Facilitan:

- Detección visual de outliers
- Comparación de escalas
- Identificación de dispersiones anómalas

Gráfico de barras por categoría

Responde a una pregunta de negocio:

¿Qué categorías concentran mayor volumen de ventas?

Heatmap de correlación

Ayuda a:

- Detectar relaciones lineales
- Anticipar multicolinealidad
- Comprender dependencias entre métricas financieras

Scatter ventas vs ingresos

Permite:

- Validar coherencia del cálculo de ingresos
- Observar patrones por categoría
- Detectar puntos atípicos multivariados

El uso de **Plotly** agrega interactividad, alineándose con estándares actuales de dashboards analíticos.

5. Limpieza de datos: garantizar calidad analítica

La limpieza se realiza de forma **ordenada y explícita**:

1. Eliminación de duplicados

Evita sesgos en estadísticas y modelos.

2. Conversión de valores inválidos (-1) a NaN

Permite que pandas y numpy manejen correctamente los faltantes.

3. Imputación por media

Elegida por:

- Simplicidad
- Interpretabilidad
- Coherencia con un ejercicio introductorio

La imputación se realiza **solo sobre columnas numéricas**, respetando la semántica de los datos categóricos.

6. Tratamiento de outliers: winsorización

En lugar de eliminar filas (lo que reduce información), se aplica **winsorización**:

- Se limitan los valores extremos al percentil 1% y 99%
- Se conserva el tamaño del dataset
- Se reduce el impacto de valores aberrantes en estadísticas y modelos

Esta técnica es especialmente apropiada en variables financieras como `ventas` e `ingresos_estim`.

7. Normalización: preparar datos para modelado

Se aplican **dos estrategias complementarias**, ambas habituales en pipelines reales:

Normalización Z-score

$$z = \frac{x - \mu}{\sigma} \quad z = \frac{x - \mu}{\sigma}$$

- Centra en 0
- Escala por desviación estándar
- Útil para modelos sensibles a la escala (regresión, clustering)

Normalización robusta

$$z_{\text{robusto}} = \frac{x - \text{mediana}}{\text{IQR}} \quad z_{\text{robusto}} = \frac{x - \text{mediana}}{\text{IQR}}$$

- Menos sensible a outliers

- Preferida cuando las distribuciones no son normales

Comparar ambas permite **entender el impacto del método elegido**.

8. Guardado de artefactos: reproducibilidad

Se guardan tres datasets:

- Dataset limpio
- Dataset normalizado con Z-score
- Dataset normalizado robusto

Esto replica una práctica real de proyectos profesionales:

- Auditoría
 - Versionado
 - Reutilización en etapas posteriores
-

9. Salida final y métricas agregadas

La salida final resume el pipeline mediante:

- Medias finales
- Desviaciones estándar

Imprimirlas en un formato **exacto y determinístico** permite:

- Evaluación automática
 - Comparación entre ejecuciones
 - Validación de reproducibilidad
-

10. Registro de ejecución

El log final incluye:

- Fecha de ejecución
- Seed utilizada
- Cantidad final de filas
- Columnas numéricas procesadas

Este registro es fundamental en:

- Pipelines productivos
- Auditorías
- Experimentos controlados

Conclusión

Esta práctica integra EDA, limpieza, visualización y preparación de datos en un flujo coherente y profesional, simulando de forma realista el trabajo cotidiano de un Data Scientist.

No se trata solo de “hacer gráficos”, sino de tomar decisiones justificadas en cada etapa, garantizando calidad, reproducibilidad y trazabilidad del análisis.

9 - Introducción al EDA reproducible en Python

Ejercicio para aplicar técnicas de optimización en NumPy mediante la elección adecuada de dtypes, uso de vistas y profiling simple para mejorar el rendimiento y reducir el consumo de memoria.

Exploración inicial, calidad de datos y visualización básica

Esta unidad introduce el concepto de Exploratory Data Analysis (EDA) como el primer paso fundamental en cualquier proyecto de Data Science. El foco está en aprender a entender el dataset antes de transformarlo, identificando problemas de calidad, observando distribuciones y generando visualizaciones básicas.

Se trabaja con un dataset simulado de negocio, se aplican análisis descriptivos simples y se producen salidas reproducibles mediante el control de una *seed*.

Esta unidad prepara el terreno para prácticas posteriores de limpieza, imputación, normalización y pipelines reproducibles.

Objetivos de aprendizaje

Al finalizar la unidad, el estudiante podrá:

- Comprender qué es el EDA y su rol en un proyecto de datos.
- Explorar la estructura básica de un dataset.
- Interpretar estadísticos descriptivos.
- Detectar valores faltantes, duplicados y posibles outliers.
- Generar visualizaciones básicas para entender distribuciones y relaciones.
- Ejecutar análisis reproducibles usando una semilla fija.
- Generar una salida final clara y resumida del análisis.

1. Contexto y motivación

En proyectos reales, los datos rara vez están listos para modelar. Antes de construir modelos o aplicar algoritmos, es necesario:

- Entender qué variables existen.
- Detectar errores o valores extraños.
- Comprender cómo se distribuyen los datos.
- Identificar relaciones entre variables.

Este proceso se llama Exploratory Data Analysis (EDA).

El objetivo del EDA no es modelar, sino comprender los datos.

2. Dataset simulado: control y aprendizaje

El dataset representa ventas de una empresa con variables típicas de negocio:

Variables categóricas

- `categoria`

- region

Variables numéricas

- ventas
- unidades
- margen_pct
- dias_stock
- satisfaction

Variable temporal

- fecha

El dataset incluye intencionalmente:

- Algunos valores faltantes
- Algunos duplicados
- Distribuciones no perfectamente normales

Esto permite practicar detección de problemas sin depender de fuentes externas.

3. ¿Qué es el EDA?

El EDA es el proceso de examinar un dataset para responder preguntas básicas:

- ¿Cuántas filas y columnas hay?
- ¿Qué tipo de variables existen?
- ¿Qué valores toman?
- ¿Hay datos faltantes?
- ¿Existen valores extremos?
- ¿Cómo se relacionan las variables?

Es un proceso exploratorio, no definitivo.

4. Estructura del dataset

El primer paso es entender la estructura general.

Conceptos clave:

Dimensión del dataset

- Número de filas: observaciones.
- Número de columnas: variables.

Tipos de datos

Cada columna tiene un tipo:

- Numérico: números continuos o discretos.
- Categórico: etiquetas o grupos.
- Temporal: fechas o timestamps.

Comprender los tipos es esencial para decidir qué análisis o gráficos aplicar.

5. Estadísticos descriptivos

Los estadísticos descriptivos resumen el comportamiento de una variable.

Conceptos principales:

Media

Promedio de los valores.

Indica el centro de la distribución, pero es sensible a outliers.

Mediana

Valor central de la distribución ordenada.

Es más robusta frente a valores extremos.

Desviación estándar

Mide cuánto se dispersan los datos alrededor de la media.

Alta desviación:

→ Datos muy dispersos.

Baja desviación:

→ Datos concentrados.

Percentiles

Indican la posición relativa dentro de la distribución.

Ejemplos:

- Percentil 25: el 25% de los valores es menor o igual.
- Percentil 75: el 75% de los valores es menor o igual.

Sirven para detectar posibles outliers.

6. Detección de problemas de calidad

Antes de cualquier transformación, se revisan problemas típicos.

6.1 Valores faltantes

Pueden aparecer como:

- NaN
- Valores especiales (como -1 o 999)

Problemas que generan:

- Sesgos en estadísticas.
- Errores en modelos.

El primer paso es **contarlos y entender su magnitud**.

6.2 Duplicados

Filas repetidas pueden:

- Inflar estadísticas.

- Sesgar modelos.

Se deben identificar y cuantificar.

Pregunta clave:

¿Qué porcentaje del dataset está duplicado?

6.3 Valores extremos (outliers)

Son valores muy alejados del resto.

Pueden representar:

- Errores de carga.
- Eventos reales pero raros.

Se detectan observando:

- Percentiles extremos.
 - Boxplots.
 - Histograma.
-

7. Visualizaciones básicas para EDA

Las visualizaciones permiten detectar patrones que no se ven en tablas.

7.1 Histograma

Muestra la distribución de una variable numérica.

Permite ver:

- Simetría o asimetría.
- Colas largas.
- Posibles outliers.

7.2 Boxplot

Resume la distribución en cinco números:

- Mínimo
- Q1 (percentil 25)
- Mediana
- Q3 (percentil 75)
- Máximo

También muestra visualmente los outliers.

7.3 Gráfico de barras

Se usa para variables categóricas.

Responde preguntas como:

- ¿Qué categoría vende más?
- ¿Qué región tiene mayor volumen?

7.4 Scatter plot (gráfico de dispersión)

Muestra la relación entre dos variables numéricas.

Permite:

- Detectar correlaciones.
 - Ver patrones.
 - Identificar outliers multivariados.
-

8. Concepto de reproducibilidad

En Data Science, los resultados deben ser **repetibles**.

Dos ejecuciones del mismo código deben producir el mismo resultado.

Para lograrlo:

- Se fija una **seed** para los generadores aleatorios.
- Se evita introducir procesos no determinísticos.
- Se ejecuta todo el flujo de principio a fin.

Esto es clave para:

- Auditoría.
- Comparación de resultados.
- Trabajo en equipo.

9. Salida final resumida

Al final del EDA se debe producir un resumen claro:

Ejemplos de métricas finales:

- Cantidad de filas y columnas.
- Porcentaje de valores faltantes.
- Cantidad de duplicados.
- Media de variables principales.
- Desviación estándar.

Este resumen permite:

- Documentar el estado del dataset.
- Comparar ejecuciones.

- Validar la reproducibilidad.

10. Registro de ejecución

Cada análisis debe registrar información básica:

- Fecha de ejecución.
- Seed utilizada.
- Tamaño final del dataset.
- Variables analizadas.

Esto convierte el análisis en un proceso trazable.

Conclusión conceptual

El EDA es la etapa donde el científico de datos:

- Comprende el dataset.
- Detecta problemas de calidad.
- Observa distribuciones y relaciones.
- Toma decisiones informadas para pasos posteriores.

Esta unidad se enfoca en **explorar y diagnosticar**, sin transformar aún los datos de forma compleja.

Las unidades siguientes incorporarán:

- Limpieza estructurada.
- Imputación.
- Tratamiento de outliers.
- Normalización.
- Pipelines reproducibles.

10 - Flujo reproducible de EDA en Jupyter/Colab (archivos de ejemplo)

Ejercicio de programación para practicar la estructuración de un flujo reproducible de análisis exploratorio de datos (EDA) en notebooks, simulando la limpieza y exportación de resultados. Se enfoca en aplicar buenas prácticas para notebooks y Google Colab, incluyendo manejo de datos ausentes y exportación de resultados.

Fundamentación teórica de la práctica

EDA avanzado, limpieza, imputación y normalización con Python

1. Contexto y objetivo general

En proyectos reales de Data Science, **la mayor parte del tiempo no se dedica al modelado**, sino a **comprender, limpiar y preparar los datos**.

El objetivo de esta práctica es simular un **flujo de trabajo profesional de Exploratory Data Analysis (EDA)** sobre un dataset de negocio realista, incorporando:

- Análisis exploratorio estructurado
- Visualizaciones interactivas
- Identificación de problemas de calidad de datos
- Limpieza, imputación y tratamiento de outliers
- Normalización de variables numéricas
- Generación de salidas reproducibles

Todo el proceso se diseña para ser **determinístico, reproducible y ejecutable en una única celda**, tal como se espera en entornos productivos o pipelines automatizados.

2. Dataset simulado: realismo y control experimental

El dataset representa **ventas de una empresa multirubro**, con variables típicas de un sistema comercial:

- Variables **categorías reales**:
- `categoria` (Alimentos, Bebidas, Electrónica, etc.)
- `region` (Norte, Sur, Este, Oeste, Centro)

- Variables **numéricas de negocio**:
- ventas, unidades, margen_pct, dias_stock, satisfaction, ingresos_estim
- Variables **temporales**:
- fecha

Además, el dataset incorpora **problemas comunes del mundo real** de forma controlada:

- Valores faltantes codificados como -1
- Duplicados
- Outliers extremos
- Distribuciones no normales (ventas log-normales)

Esto permite practicar técnicas de EDA **sin depender de fuentes externas** y sin introducir incertidumbre sobre la calidad del dato original.

3. EDA básico: entender antes de transformar

El primer bloque del EDA responde a una pregunta clave:

¿Qué datos tengo y en qué estado están?

Por eso se analizan:

- **Estructura del dataset**: columnas y tipos de datos
- **Estadísticos descriptivos**: media, desviación, percentiles
- **Valores inválidos**: conteo de -1 por columna
- **Duplicados**: impacto en el tamaño efectivo del dataset
- **Percentiles extremos**: para anticipar la presencia de outliers

Este paso es crítico porque **ninguna transformación posterior es válida si no se comprende el estado inicial de los datos**.

4. Visualizaciones interactivas: detectar patrones y anomalías

Las visualizaciones permiten **detectar problemas que no se ven en tablas**. En esta práctica se utilizan gráficos coherentes con análisis profesional:

Histograma de ventas

Permite observar:

- Asimetría
- Cola larga
- Impacto de outliers
- Distribución real de ingresos

Boxplots de variables numéricas

Facilitan:

- Detección visual de outliers
- Comparación de escalas
- Identificación de dispersiones anómalas

Gráfico de barras por categoría

Responde a una pregunta de negocio:

¿Qué categorías concentran mayor volumen de ventas?

Heatmap de correlación

Ayuda a:

- Detectar relaciones lineales
- Anticipar multicolinealidad
- Comprender dependencias entre métricas financieras

Scatter ventas vs ingresos

Permite:

- Validar coherencia del cálculo de ingresos
- Observar patrones por categoría
- Detectar puntos atípicos multivariados

El uso de **Plotly** agrega interactividad, alineándose con estándares actuales de dashboards analíticos.

5. Limpieza de datos: garantizar calidad analítica

La limpieza se realiza de forma **ordenada y explícita**:

1. Eliminación de duplicados

Evita sesgos en estadísticas y modelos.

2. Conversión de valores inválidos (-1) a NaN

Permite que pandas y numpy manejen correctamente los faltantes.

3. Imputación por media

Elegida por:

- Simplicidad
- Interpretabilidad
- Coherencia con un ejercicio introductorio

La imputación se realiza **solo sobre columnas numéricas**, respetando la semántica de los datos categóricos.

6. Tratamiento de outliers: winsorización

En lugar de eliminar filas (lo que reduce información), se aplica **winsorización**:

- Se limitan los valores extremos al percentil 1% y 99%
- Se conserva el tamaño del dataset
- Se reduce el impacto de valores aberrantes en estadísticas y modelos

Esta técnica es especialmente apropiada en variables financieras como `ventas` e `ingresos_estim`.

7. Normalización: preparar datos para modelado

Se aplican **dos estrategias complementarias**, ambas habituales en pipelines reales:

Normalización Z-score

$$z = \frac{x - \mu}{\sigma} \quad z = \frac{x - \mu}{\sigma}$$

- Centra en 0
- Escala por desviación estándar
- Útil para modelos sensibles a la escala (regresión, clustering)

Normalización robusta

$$z_{\text{robusto}} = \frac{x - \text{mediana}}{\text{IQR}_{\text{robusto}}} = \frac{x - \text{mediana}}{\text{IQR}}$$

- Menos sensible a outliers
- Preferida cuando las distribuciones no son normales

Comparar ambas permite **entender el impacto del método elegido**.

8. Guardado de artefactos: reproducibilidad

Se guardan tres datasets:

- Dataset limpio
- Dataset normalizado con Z-score
- Dataset normalizado robusto

Esto replica una práctica real de proyectos profesionales:

- Auditoría
 - Versionado
 - Reutilización en etapas posteriores
-

9. Salida final y métricas agregadas

La salida final resume el pipeline mediante:

- Medias finales
- Desviaciones estándar

Imprimirlas en un formato **exacto y determinístico** permite:

- Evaluación automática
- Comparación entre ejecuciones
- Validación de reproducibilidad

10. Registro de ejecución

El log final incluye:

- Fecha de ejecución
- Seed utilizada
- Cantidad final de filas
- Columnas numéricas procesadas

Este registro es fundamental en:

- Pipelines productivos
- Auditorías
- Experimentos controlados

Conclusión

Esta práctica integra **EDA, limpieza, visualización y preparación de datos** en un flujo coherente y profesional, simulando de forma realista el trabajo cotidiano de un Data Scientist.

No se trata solo de “hacer gráficos”, sino de **tomar decisiones justificadas en cada etapa**, garantizando calidad, reproducibilidad y trazabilidad del análisis.

11 - Indexación avanzada y manejo de slices en arrays

Ejercicio para practicar técnicas de indexación avanzada en arrays: selección con boolean masks, fancy indexing y manejo correcto de vistas y copias.

Ejercicio: Suma y multiplicación con broadcasting

En este ejercicio, recibirás dos arrays numéricos representados como listas de enteros. El primer array puede ser de una dimensión o dos dimensiones, y el segundo array puede ser de una dimensión o un escalar.

Tu tarea es implementar dos operaciones usando broadcasting y vectorización:

- Sumar ambos arrays usando broadcasting.

- Multiplicar ambos arrays usando broadcasting.

No debes usar bucles explícitos para realizar estas operaciones; en su lugar, utiliza las reglas de broadcasting para operar directamente sobre los arrays.

Formato de entrada

- La primera línea contiene dos enteros separados por espacio: n m que representan las dimensiones del primer array.
- Si m es 0, el array es unidimensional con n elementos.
- Si m es mayor que 0, el array es bidimensional con n filas y m columnas.
- Las siguientes n líneas contienen los elementos del primer array:
- Si m es 0, cada línea contiene un solo entero.
- Si m es mayor que 0, cada línea contiene m enteros separados por espacios.
- La siguiente línea contiene un entero k que indica el tamaño del segundo array:
- Si k es 1, el segundo array es un escalar.
- Si k es mayor que 1, el segundo array es unidimensional con k elementos.
- La siguiente línea contiene los elementos del segundo array separados por espacios.

Formato de salida

- Imprime primero el resultado de la suma, línea por línea, con los elementos separados por espacios.
- Luego imprime una línea en blanco.
- Finalmente, imprime el resultado de la multiplicación, con el mismo formato.

Ejemplo

Entrada:

```
2 31 2 34 5 6310 20 30
```

Salida:

```
11 22 3314 25 36
10 40 9040 100 180
```

Explicación: El primer array es 2x3, el segundo es un array 1D con 3 elementos. Se aplica broadcasting para sumar y multiplicar cada fila del primer array con el segundo array.

Restricciones

- $1 \leq n \leq 100$
- $0 \leq m \leq 100$
- $1 \leq k \leq 100$

- Elementos enteros en rango $[-10^6, 10^6]$
-

Código inicial (Python 3)

```
python
import numpy as np
# =====# EJERCICIO 1:
# Suma y multiplicación con broadcasting#
# =====#
def suma_y_multiplicacion_broadcast(arr1, arr2):    """ Realiza: - Suma con
broadcasting - Multiplicación con broadcasting Retorna (suma,
multiplicacion)
arr1 y arr2 pueden ser 1D o 2D, siempre compatibles para broadcasting.    """#
Completar

# =====# EJERCICIO 2:
# Manipulación avanzada de subarrays e indexación#
# =====#
def modify_array_inplace(arr, threshold):    """ Modifica arr IN-PLACE: 1)
Elementos > threshold => multiplicar por 2 2) Elementos en posiciones impares
=> sumar 10    """#Completar
    return arr
```
